

Latent Liking Estimation

Tom van Hogen

Introduction

The goal of the provided scripts, which contents are the same as the code within this document, is to estimate a participant's latent liking scores, based on the behavioral data collected during the MRI experiment. These scores provide perhaps the most up-to-date representation of aesthetic liking, and could most closely track the neural activity in the brain in relation to this factor.

The behavioral data collected from the MRI experiment resulted from a continuous 1-back task, where the subject is asked to give a preference of one out of two paintings, i.e., the currently presented painting, or the previous one. This can be viewed as a logistic regression problem, which estimates the probability of one painting being chosen over the other one. Going further, we can supply each painting with a latent score, and translate the probability to a distance between each painting's latent score, where a high probability of painting X being picked over Y translates to a greater distance in their respective latent scores. This idea comes from the Bradley-Terry model and is outlined in greater detail in the manuscript.

In order to facilitate this model, and to get a good estimate of the estimation uncertainty, we made use of *JAGS*, which stands for *Just Another Gibbs Sampler*. In essence, we are applying Bayesian Logistic Regression, but with the Bradley-Terry model underlying it. For more information on the *JAGS* package, see: <https://mcmc-jags.sourceforge.io/>.

The code performs the following steps:

1. Preprocessing the data in order to create appropriate input data for the JAGS model, and to prepare datasets for later analysis.
2. Setting up the JAGS model and specifying its structure, as well as providing the Markov chains with initial values.
3. Running the JAGS model to estimate each subject's latent liking scores for all paintings.
4. Running analysis on the predictive validity of the model. Here we check how well the latent liking scores predict the final ranking of paintings, which results from a ranking task done after the MRI session.

5. K-fold validation. In order to check model performance and how well it generalizes, we train the model on one part of the subject's data and test it against another part (which is unseen data). This is a lengthy procedure as we need to train and test a model k (default = 10) times for each of the 44 subjects.
6. Finally, we gather all results, run additional analysis and plot everything in order to get a nice view of the model's performance and ability to predict the final ranking data.

1 Preprocessing the Data

This part deals with the preprocessing of the data, in order to prepare everything for the JAGS model. The data that we are going to process can be found in the *data* folder, which is expected to be one level above the working directory. Within the *data* folder there are subject-specific directories, which further contain three more directories. We are for now just interested in the one called *mri_runs_behavioral*.

First we import the packages we need.

```
library(dplyr)
library(tidyr)
library(readr)
library(readxl)
library(writexl)
library(runjags)
library(bayestestR)
library(ggplot2)
library(caret)
library(ggpubr)
library(effectsize)

# Load functions
source(file.path("utils", "jags_functions.R"))
```

Then, we need to set up a couple of things. First of all, we need to specify the location of the *data* directory. Then, we get a vector of all subject directories found within this folder, as well as a vector containing just the subject labels. We also specify the location where important variables from this script will be stored.

```
#### Get data file paths across all subjects ####
# Location of data files
data_path <- file.path("../", "data") # using file.path to make it platform
↪ independent
```

```

# Get subject directories
subject_dirs <- list.dirs(data_path, recursive=FALSE)
subject_dirs_rel <- list.dirs(data_path, full.names = F, recursive = F) #
  ↳ just subject names

# location for storing the datasets
datasets_var_path <- file.path("varstore", "datasets.RData")
matrices_var_path <- file.path("varstore", "matrices.RData")
ranking_var_path <- file.path("varstore", "ranking_data.RData")
rating_var_path <- file.path("varstore", "rating_data.RData")

# store total number of subjects
n_subjects <- length(subject_dirs)

```

Then using the behavioral data files of each subject, we import these into separate dataframes and store them in a list object called *datasets*, labeled by each subject. We also take care to remove the jittered trials, as these are empty trials and offer no information.

```

#### Pre-processing and preparation of the behavioral dataset ####
datasets <- list()

# Loop across participants
for (i in 1:n_subjects){
  # Path for subject's folder containing behavioral data collected during MRI
  ↳ runs
  sub_data_path <- file.path(subject_dirs[i], "mri_runs_behavioral")

  # Within folder list all datafiles (one .csv file per run)
  sub_data_files <- list.files(path = sub_data_path, full.names = TRUE)
  n_runs <- length(sub_data_files)

  # Initialize empty list
  temp_all_data <- list()
  # Read datafiles and combine into single dataframe
  for (j in 1:length(sub_data_files)){
    # Read the CSV file
    data <- read.csv(sub_data_files[j])
    # append data to temporary container list
    temp_all_data <- append(temp_all_data, list(data))
  }
}

```

```

# Combine all separate runs into one single dataframe
sub_all_data <- do.call(rbind, temp_all_data)

# Remove jittered trials (that are missing values in cat1 or cat2)
last_col <- ncol(sub_all_data)
sub_all_data <- sub_all_data[sub_all_data[, last_col] != "" & sub_all_data[,
↪ (last_col-1)] != "", ]

# Store in list for all participants
datasets[[subject_dirs_rel[i]]] <- sub_all_data
}

```

We can now use these imported datasets to start building a data structure that we can use as input data for the JAGS model.

The individual datasets are transformed to two types of matrices: M_{pref} and M_n . Each matrix has 36 rows and 36 columns, corresponding with the 36 paintings. Each cell represents a pair of paintings which are considered together during a trial of the 1-back MRI task. Cell values represent the amount of preferences for a specific painting in a pair (in the case of M_{pref}), or the total number of pairings (in the case of M_n). For example, if the cell belonging to painting 5 (row) and painting 13 (column) has the value 3, it means that painting 5 was preferred 3 times over painting 13 (in the case of M_{pref}), or that the pairing has occurred 3 times in total (in the case of M_n).

To aid readability, a function to create the matrices is sourced from *utils/jags_functions.R*.

```

matrices <- list() # initialize list
for (i in 1:n_subjects){
  subject <- subject_dirs_rel[i]
  sub_all_data <- datasets[[subject]]
  matrices[[subject]] <- create_matrices(sub_all_data) # reusable function
↪ from utils/jags_functions.R
}

```

Other preprocessing steps deal with the pre-mri liking data, and the final ranking data.

```

true_ranks_subs <- list() # Initialize list that contains 'true' ranking data
↪ for all subjects, which is the final ranking
ratings <- list() # initialize list that will contain pre-mri rating data per
↪ subject

```

```

for (i in 1:n_subjects){
  # subject id
  subject <- subject_dirs_rel[i]
  # Location of ranking data
  ranking_file <- file.path(subject_dirs[i], "final_ranking",
↪  "post_mri_ranking.xlsx")

  # Read datafile
  ranking_data <- read_excel(ranking_file)

  # Extract ranking vector, which is a sequence of painting ID's, ranked from
↪  highest to lowest rank
  true_ranks_vec <- ranking_data$MRI_code

  # Drop all columns but MRI_code and rank, sort by MRI_code, rename columns
↪  to ID and Ranking, and convert to data.frame
  ranking_data <- ranking_data %>%
    select(MRI_code, rank) %>%
    arrange(MRI_code) %>%
    rename(ID=MRI_code, Ranking=rank) %>%
    as.data.frame()

  ranking_entry <- list(data=ranking_data, ranking_vector=true_ranks_vec)
  true_ranks_subs[[subject]] <- append(true_ranks_subs[[subject]],
↪  ranking_entry)

  # Get pre-mri ratings
  rating_file <- file.path(subject_dirs[i], "pre_mri_rating",
↪  "pre_mri_rating.xlsx")
  rating_data <- read_excel(rating_file) %>%
    select(Label, liking_scores) %>%
    rename(MRI_code=Label)

  ratings[[subject]] <- rating_data
}

```

Finally, we store all data in the *varstore* directory.

```

# Store datasets
save(datasets, file=datasets_var_path)
save(matrices, file=matrices_var_path)

```

```
save(true_ranks_subs, file=ranking_var_path)
save(ratings, file=rating_var_path)
```

2 JAGS Set-Up and Initialization

Before we can run the JAGS model, we need to set up important parameters, specify the model structure, specify the number of Markov chains, and initialize parameter values for the Markov chains.

```
#### JAGS MODEL ####
modelString <- "
  model{
    sf <- 10 # scale factor

    for(i in 1:nPaintings){
      for(j in 1:nPaintings){
        # Likelihood
        y[i,j] ~ dbin(theta[i,j], n[i,j])
        logit(theta[i,j]) <- liking[i] - liking[j]
      }
    }
    # priors
    # Uninformative priors for all paintings except the first.
    for (k in 2:nPaintings){
      liking[k] <- liking_[k] * sf # scaled score
      liking_[k] ~ dbeta(a[k], b[k])
      a[k] <- ifelse((mu[k] * kappa) < 1, 1, mu[k] * kappa)
      b[k] <- ifelse((kappa - a[k]) < 1, 1, kappa - a[k])

      # Priors over reparameterizations
      mu[k] ~ dbeta(1,1)
    }
    liking[1] <- sf/2 # fix one value in middle of scale to make model
    ↪ identifiable
      # Hyperprior over kappa
      kappa ~ dunif(0.001, 100) # We assume similar confidence for all
    ↪ comparisons
  }"

# Write model to local textfile
```

```

writeLines(modelString, con = "models/ranking_logregres_bin_reparam.txt")

#### JAGS Initialization ####
# Set seed
set.seed(1234)

# number of chains
nChains <- 3

# Generate initial values list for three chains
inits <- generate_initial_values(nChains) # external function, source
↳ utils/jags_functions.R if necessary.

```

3 Running JAGS

It is time to run the model. We set the seed to 1234 and define a variable that will store each subject's output from the JAGS model.

```

# set seed
set.seed(1234)

# Initialize list variable which will contain outputs across subjects
jags_output_subs <- list()

```

Next, we iterate over each subject and run the model on the matrices we created earlier and using the initial values we created. This will take some time to finish running. We print out the results of the first participant to give an example of the model output. If you have run it before, there is an option to load the previous results. We will load it here.

```

# Set parameter for loading previous results, or re-running the code
load_jags <- TRUE

if (load_jags==TRUE){
  load(file.path("varstore", "jags_output_all_subs.RData"))
} else {
  # iterate over subject
  for (i in 1:n_subjects){
    # Pick subject and create string object
    subject <- subject_dirs_rel[i]

```

```

# Print status message
print(paste("Running JAGS for:", subject))

# Retrieve subject's data matrices, which will be used as input data.
M_pref <- matrices[[subject]]$M_pref
M_n <- matrices[[subject]]$M_n

# Prepare the data
dataList <- list(y = M_pref,
                 n = M_n,
                 nPaintings = 36)

# Model parameters
pars <- c("liking")
samples <- 10000
burnin <- 1000
nAdapt <- 1000

output <- run.jags(model="models/ranking_logregres_bin_reparam.txt",
                   monitor=pars,
                   data=dataList,
                   n.chains = nChains,
                   inits=inits,
                   burnin=burnin,
                   sample=samples,
                   adapt=nAdapt,
                   method="rjparallel"
)

# Store outputs
jags_output_subs[[subject]] <- output
}
}

```

Let us examine the result from the first subject, just to see the kind of output we get.

```
jags_output_subs$`sub-e01`
```

JAGS model summary statistics from 30000 samples (chains = 3; adapt+burnin = 2000):

	Lower95	Median	Upper95	Mean	SD	Mode	MCerr	MC%ofSD
liking[1]	5	5	5	5	0	5	--	--
liking[2]	9.2119e-06	0.56258	1.5082	0.64074	0.45945	--	0.0083466	1.8
liking[3]	1.253	2.6348	4.052	2.6492	0.71882	--	0.012426	1.7
liking[4]	5.8302	7.3883	9.0693	7.406	0.83526	--	0.015813	1.9
liking[5]	0.000035183	0.45638	1.3233	0.53927	0.40732	--	0.0070743	1.7
liking[6]	0.94673	2.1058	3.2673	2.1034	0.58916	--	0.010432	1.8
liking[7]	9.2583e-06	0.15278	0.62241	0.21331	0.20104	--	0.003241	1.6
liking[8]	2.0623	3.2879	4.5432	3.293	0.63496	--	0.011635	1.8
liking[9]	5.3741	6.7718	8.387	6.8144	0.77218	--	0.012976	1.7
liking[10]	7.6811	9.2692	9.9999	9.0967	0.723	--	0.016403	2.3
liking[11]	3.1749	4.3664	5.5587	4.3666	0.61189	--	0.0098206	1.6
liking[12]	1.021	2.1206	3.2658	2.1276	0.57514	--	0.0095051	1.7
liking[13]	3.2236	4.4574	5.7258	4.4601	0.63757	--	0.011957	1.9
liking[14]	0.60002	1.7307	2.8615	1.7353	0.57537	--	0.0098194	1.7
liking[15]	0.000021428	0.33839	1.0808	0.41846	0.33872	--	0.0059188	1.7
liking[16]	6.7222	8.2911	9.9867	8.2746	0.90919	--	0.019734	2.2
liking[17]	1.8105	3.027	4.2778	3.0308	0.63377	--	0.0099333	1.6
liking[18]	1.7339	2.8109	3.8807	2.8176	0.55173	--	0.0095344	1.7
liking[19]	3.7208	4.7724	5.881	4.7725	0.55005	--	0.0093043	1.7
liking[20]	3.8064	5.1139	6.402	5.1233	0.6637	--	0.010176	1.5
liking[21]	9.9987e-06	0.29288	1.015	0.37687	0.32178	--	0.005605	1.7
liking[22]	3.324	4.489	5.7157	4.492	0.61088	--	0.0093085	1.5
liking[23]	4.579e-06	0.79569	1.8464	0.85835	0.5437	--	0.010573	1.9
liking[24]	8.2848	9.5166	10	9.3681	0.54257	--	0.0115	2.1
liking[25]	0.9914	2.1226	3.2345	2.1288	0.5735	--	0.0097153	1.7
liking[26]	0.80714	1.8729	2.937	1.8793	0.54704	--	0.0095681	1.7
liking[27]	2.859	4.1112	5.3606	4.1273	0.6385	--	0.010913	1.7
liking[28]	2.751	3.9047	5.1122	3.9109	0.60566	--	0.011169	1.8
liking[29]	0.000057235	0.85283	1.8204	0.8954	0.51983	--	0.0094229	1.8
liking[30]	2.9808	4.0898	5.2827	4.0976	0.58972	--	0.010703	1.8
liking[31]	5.3633	6.9565	8.5587	6.9918	0.82439	--	0.015731	1.9
liking[32]	5.9892	7.8879	9.9868	7.8618	1.0908	--	0.024415	2.2
liking[33]	4.9473	6.4015	8.1456	6.44	0.81586	--	0.013126	1.6
liking[34]	0.000056531	0.53041	1.433	0.60619	0.43699	--	0.0080114	1.8
liking[35]	3.146	4.2356	5.257	4.2329	0.53804	--	0.0095667	1.8
liking[36]	3.795	4.9395	6.0988	4.9476	0.58214	--	0.0090777	1.6

	SSeff	AC.10	psrf
liking[1]	--	--	--
liking[2]	3030	0.15342	1.0006

liking[3]	3346	0.13107	1.0001
liking[4]	2790	0.17976	1.0026
liking[5]	3315	0.13944	1.0006
liking[6]	3189	0.14687	1.0007
liking[7]	3848	0.099566	1.0009
liking[8]	2979	0.15108	1.0001
liking[9]	3541	0.13099	1.0014
liking[10]	1943	0.28112	1.0006
liking[11]	3882	0.11151	1.0002
liking[12]	3661	0.12307	1.0003
liking[13]	2843	0.18167	1.001
liking[14]	3433	0.14208	1.0005
liking[15]	3275	0.14102	1.0004
liking[16]	2123	0.25752	1.0029
liking[17]	4071	0.099154	1.0004
liking[18]	3349	0.1479	1.0002
liking[19]	3495	0.13846	1.0005
liking[20]	4254	0.10231	1.0002
liking[21]	3296	0.13113	1.0007
liking[22]	4307	0.097238	1.0004
liking[23]	2644	0.18859	1.0001
liking[24]	2226	0.25601	1.0015
liking[25]	3485	0.13523	1.0006
liking[26]	3269	0.13693	1.0002
liking[27]	3423	0.1462	1.0002
liking[28]	2941	0.1567	1.0006
liking[29]	3043	0.15791	1.0009
liking[30]	3036	0.15317	1.0003
liking[31]	2746	0.18187	1.0021
liking[32]	1996	0.26759	1.0006
liking[33]	3863	0.09797	1
liking[34]	2975	0.15016	1.0006
liking[35]	3163	0.11546	1.0003
liking[36]	4113	0.10164	1.001

Total time taken: 12.6 seconds

As you can see, the first painting does not show any variation. This is because we fixed its score to the value of 5. All scores are relative to each other, and without fixing one value, there are infinite ways to solve the estimation problem.

Finally, we save the data in the *varstore* directory under filename *jags_output_all_subs.RData*.

```
# Save the data to local variable
if (load_jags==FALSE){
  save(jags_output_subs, file=file.path("varstore",
    ↪ "jags_output_all_subs.RData"))
}
```

4 Model Performance Check

4.1 Predictive Validity

Having run the model for each subject, and having an estimation of their latent liking scores for the paintings, we would like to know how well it can predict the results from the final ranking task.

In order to do this, we take the posterior samples (JAGS output) of the latent liking scores from each subject, rank-transform them, and apply a Spearman correlation test together with the final ranking data. In this way, we end up with a posterior distribution of the correlation between latent liking scores and the final ranking data, which is an indication of the predictive validity.

We do some set-up and initialization. Also creating folders if they do not already exist.

```
# initialize variables
subs_validity <- list() # contains per subject information on the pred. val.
  ↪ posterior distribution.

n_samples <- jags_output_subs$`sub-e01`$sample * nChains
cor_samples_df <- data.frame("sub-e01"=rep(NA, n_samples)) # will contain
  ↪ correlations across model samples
liking_stages_mat <- matrix(NA,
  nrow = n_subjects*36,
  ncol = 5) # Contains liking scores from all three
  ↪ stages

# Create folder for data and figure storage
results_dir <- file.path("../", "results")
ifelse(!dir.exists(results_dir),
  dir.create(results_dir),
  "Directory Exists")
```

```
[1] "Directory Exists"
```

```

ifelse(!dir.exists(file.path(results_dir, "figures")),
      dir.create(file.path(results_dir, "figures")),
      "Directory Exists")

```

```
[1] "Directory Exists"
```

Then, we iterate over each subject and correlate each posterior sample of the latent liking scores with the final ranking data. We also store a subject's results in a dataframe containing an overview of the liking scores across all stages of the experiment session, and write this to an excel file called *liking_three_stages.xlsx*. Furthermore, we store information regarding the predictive validity posterior distribution in a variable called *HDI_data*, which is stored and labeled by subject in *subs_validity*.

```

# Loop across subjects
for (i in 1:n_subjects){
  # string object for picking subject
  subject_string <- subject_dirs_rel[i]
  # Extract MCMC samples for latent ranks
  liking_samples <- combine.mcmc(jags_output_subs[[subject_string]])
  # Calculate correlation for each MCMC sample with the ranking task data.
  ↪ This is a measure of predictive validity.
  cor_samples <- apply(liking_samples, 1,
                      function(x) cor(rank(-x),
                      true_ranks_subs[[subject_string]]$data$Ranking,
                      method='spearman')
                      ) # Reverse rank (higher liking scores correspond to
  ↪ higher ranks. In other words, they are closer to rank 1)

  # start and end indices depends on subject number
  i_start <- (i-1)*36 + 1
  i_end <- i_start + 35
  # Enter subject number and painting ID
  liking_stages_mat[i_start:i_end, 1] <- subject_string
  liking_stages_mat[i_start:i_end, 2] <- 1:36
  # Extract and store ratings
  subject_rating <- ratings[[subject_string]]$liking_scores
  liking_stages_mat[i_start:i_end, 3] <- subject_rating
  # Extract latent liking scores and store
  subject_latent <-
  ↪ as.numeric(jags_output_subs[[subject_string]]$summary$statistics[, 'Mean'])
  liking_stages_mat[i_start:i_end, 4] <- subject_latent

```

```

# Extract rankings and store
subject_ranking <- true_ranks_subs[[subject_string]]$data$Ranking
liking_stages_mat[i_start:i_end, 5] <- subject_ranking

# Create csv file with data on subject
subject_df <- data.frame(
  label = 1:36,
  rating = subject_rating,
  latent_liking = subject_latent,
  ranking = subject_ranking
)

save_liking_path <- file.path(results_dir, subject_string,
↪  "liking_three_stages.xlsx")

# Create directory if necessary
ifelse(!dir.exists(file.path(results_dir, subject_string)),
  dir.create(file.path(results_dir, subject_string)),
  "Directory Exists")
# Write csv file
write_xlsx(subject_df, path=save_liking_path)

# Store in dataframe
cor_samples_df[subject_string] <- cor_samples

# Summarize the posterior distribution of the correlations
# First get median values
median_r <- median(cor_samples)
# Get highest density interval
HDI <- hdi(cor_samples)
# store into list
HDI_data <- list(median=median_r,
  HDI.L=as.numeric(HDI)[2],
  HDI.H=as.numeric(HDI)[3])

# Store data to list variable
subs_validity[[subject_string]] <- HDI_data
}

```

Lastly, we preprocess *liking_stages_df* for a later stage.

```

# Transform liking scores matrix to dataframe, add column names, and
↪ factorize
liking_stages_df <- as.data.frame(liking_stages_mat)

# Name columns
colnames(liking_stages_df) <- c("Subject", "PaintID", "Rating_Score",
↪ "Latent_Score", "Ranking_Score")

# Factorize
liking_stages_df <- liking_stages_df %>%
  mutate(Subject = factor(Subject),
         PaintID = factor(PaintID),
         Rating_Score = as.numeric(Rating_Score),
         Latent_Score = as.numeric(Latent_Score),
         Ranking_Score = as.numeric(Ranking_Score))

```

4.2 k-Fold Validation

In order to check model performance and how well it generalizes, we train the model on one part of the subject's data and test it against another part (which is unseen data). This is a lengthy procedure as we need to train and test a model k (default = 10) times for each of the 44 subjects.

We start with setting up important parameters. The variable *acc_mat* is a matrix that will store prediction accuracy values across subjects and folds. It has three columns: subject label, fold number, and prediction accuracy.

```

# Set parameters
# Model parameters
pars <- c("liking")
nChains <- 3
samples <- 10000
burnin <- 1000
nAdapt <- 1000

# k-fold validation parameters
k <- 10 # number of folds

# Matrix that will store the model prediction accuracy across subjects and
↪ folds.
acc_mat <- matrix(NA, nrow=length(datasets)*k, ncol = 3)

```

Next, we run the k-fold validation procedure. This is a very lengthy process which can easily take up a couple of hours. Make sure to take that into account before running. If you have run it before, there is an option to load the previous results. We will load it here.

```
# Set parameter to load previous results, or run k-fold validation procedure
load_k_fold <- TRUE

if (load_k_fold==TRUE){
  load(file.path("varstore", "k_fold_val_df.RData"))
} else {
  for (S_i in 1:length(datasets)){
    # Get subject data
    subject_string <- subject_dirs_rel[S_i]
    sub_all_data <- datasets[[subject_string]]

    for (fold in 1:k){
      # Create train and test datasets
      data <- k_fold_dataloader(sub_all_data, fold, k)
      trainset <- data$train_data
      testset <- data$test_data
      # Prepare matrices for JAGS
      mats <- create_matrices(trainset)
      M_pref <- mats$M_pref
      M_n <- mats$M_n
      # Initialize chains
      set.seed(1234)
      inits <- generate_initial_values(3)
      # Train JAGS model
      # Prepare the data
      dataList <- list(y = M_pref,
                      n = M_n,
                      nPaintings = 36)

      # Print message to console
      cat(paste("Running fold", fold, "for", subject_string, "\n\n"))
      # Run JAGS
      output <- run.jags(model="models/ranking_logregres_bin_reparam.txt",
                        monitor=pars,
                        data=dataList,
                        n.chains = nChains,
                        inits=inits,
                        burnin=burnin,
                        sample=samples,
```

```

                                adapt=nAdapt,
                                silent.jags = TRUE,
                                method="rjparallel"
                                )
# Use JAGS model to predict
# Get liking scores
latent_liking <- as.numeric(output$summary$statistics[, 'Mean'])
# then predict
preds_df <- predict_response(testset, latent_liking) # Get predictions
# Evaluate and store results
conf_mat <- confusionMatrix(preds_df$pred, preds_df$preferredStim) #
↪ Get accuracy and related measures
  mat_index_start <- S_i * k - (k-1)
  mat_index_current <- mat_index_start + (fold-1)
  acc_mat[mat_index_current,] <- c(subject_string,
                                fold,
                                ↪ as.numeric(conf_mat$overall["Accuracy"]))
}
}
}

```

Then, transform the matrix to a dataframe and store the results.

```

# Store results
if (load_k_fold==FALSE){
  # Convert to dataframe and save
  df_k_fold_acc <- data.frame(acc_mat)
  colnames(df_k_fold_acc) <- c("Subject", "Fold", "Accuracy") # name columns

  # Factorize variables
  df_k_fold_acc <- df_k_fold_acc %>%
    mutate(Subject = factor(Subject),
           Fold = factor(Fold),
           Accuracy = as.numeric(Accuracy))

  save(df_k_fold_acc, file=file.path("varstore", "k_fold_val_df.RData"))
}

# Give example of results
print(head(df_k_fold_acc))

```

	Subject	Fold	Accuracy
1	sub-e01	1	0.9354839
2	sub-e01	2	0.8620690
3	sub-e01	3	0.9000000
4	sub-e01	4	0.7931034
5	sub-e01	5	0.9615385
6	sub-e01	6	0.9259259

5 Final Analysis and Visualization

The final part of this document. We are going to gather all the results and create two final plots: one showing the posterior distribution of the predictive validity across all subjects, the other showing the k-fold validation results across all subjects and folds.

First some basic setup.

```
# Create folder for data and figure storage
figures_path <- file.path("../", "results", "figures")
ifelse(!dir.exists(figures_path),
       dir.create(figures_path),
       "Directory Exists")
```

```
[1] "Directory Exists"
```

5.1 Predictive Validity

Then, we create a dataframe called *ranked_df* which is sourced from *liking_stages_df*. The difference is that all scores across different stages are rank-transformed, if that is not already the case. We then use this to calculate the Spearman correlation between the different stages.

Important to note is that, with regards to the latent scores, the correlation here is based on single value estimates of the latent liking score, as opposed to using the full posterior distribution. We will take a look at that later.

```
# Correlations across stages
ranked_df <- liking_stages_df %>%
  # filter(Subject != "sub-n23" & Subject != "sub-n12") %>% # filter outliers
  group_by(Subject) %>%
  mutate(Rating_Score=rank(-Rating_Score),
         Latent_Score=rank(-Latent_Score))
```

```

cor_rat_ranked <- cor.test(ranked_df$Rating_Score, ranked_df$Ranking_Score,
                           method = 'spearman')
cor_rat_latent <- cor.test(ranked_df$Rating_Score, ranked_df$Latent_Score,
                           method = 'spearman')
cor_lat_ranked <- cor.test(ranked_df$Latent_Score, ranked_df$Ranking_Score,
                           method = 'spearman')

```

The most interesting result relates to how well our model corresponds to the final ranking data. We find a strong and significant correlation, $r = 0.86$, $p < 0.001$). This includes data of two outliers, which optionally can be filtered.

As mentioned before, we can also examine the mean correlation based on the full posterior distributions. Let us first check the posterior distribution using a density plot.

```

# Create long format of dataframe
cor_samples_long <- cor_samples_df %>%
  select(-"sub.e01") %>%
  pivot_longer(cols=starts_with("sub-"),
               names_to="Subject",
               values_to="Sample_r") %>%
  mutate(Subject= factor(Subject))

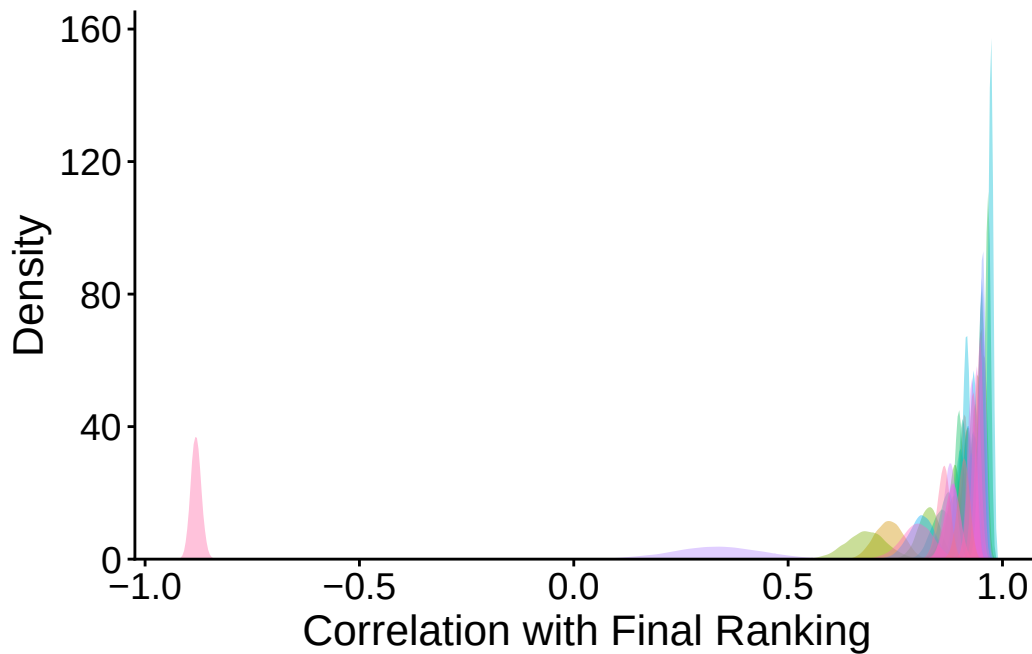
# Create plot using ggplot
pred_val_plot <- ggplot(cor_samples_long, aes(x=Sample_r, colour=Subject,
  ↪ fill=Subject)) +
  geom_density(alpha = 0.4, color=NA) +
  #geom_vline(aes(xintercept = mean_corr), color='red', linewidth = 1,
  ↪ linetype = 'dashed') +
  xlab("Correlation with Final Ranking") +
  ylab("Density") +
  theme_classic() +
  theme(legend.position = "none") +
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  theme(
    axis.title = element_text(size = 16),      # Change axis title size
    axis.text = element_text(size = 14))

# Store the plot
ggsave(filename = "predictive_validity.png", plot=pred_val_plot, path =
  ↪ figures_path, units = "px", dpi = 300)

```

Saving 1650 x 1050 px image

```
# Show plot  
pred_val_plot
```



We can see there is one, maybe two, outliers. For this reason we will do the further analysis both with and without these outliers.

We are going to quantify the correlation, and thus get a concrete measure of the predictive validity.

```
# Create long format of dataframe  
cor_samples_long <- cor_samples_df %>%  
  select(-"sub.e01") %>%  
  pivot_longer(cols=starts_with("sub-"),  
               names_to="Subject",  
               values_to="Sample_r") %>%  
  mutate(Subject= factor(Subject))  
  
# Get the average correlation  
mean_corr <- mean(cor_samples_long$Sample_r)  
# calculate the Equal-Tailed Interval (ETI) 95% credible interval  
eti_ci <- ci(cor_samples_long$Sample_r, method = "ETI")
```

```
# Also check without outliers
cor_samples_no_outliers <-
  cor_samples_long %>% filter(Subject!= "sub-n23" & Subject!= "sub-n12")

mean_corr_no_outliers <- mean(cor_samples_no_outliers$Sample_r)
eti_ci_no_outliers <- ci(cor_samples_no_outliers$Sample_r, method = "ETI")
```

As expected, we find a strong and significant correlation, $r = 0.85$, 95%CI[0.2, 0.97]. The credible interval is quite wide, due to the inclusion of outliers (which skews the distribution).

Without outliers, we find, not surprisingly, a stronger and more significant correlation, $r = 0.9$, 95%CI[0.71, 0.97].

We can conclude the model is very capable of predicting the final ranking of the paintings, based on incomplete preference data from the MRI experiment, regardless of outlier inclusion.

Let us further define a variable storing correlation statistics between liking scores from the different stages of the experiment, but grouped by subject. Additionally, we take a look at these correlations for our two outliers.

```
# Summary df
cor_summary <- cor_samples_long %>%
  group_by(Subject) %>%
  summarise(mean = mean(Sample_r),
            sd = sd(Sample_r))

# Inspect outliers
liking_stages_outliers <- ranked_df %>%
  filter(Subject == "sub-n23" | Subject == "sub-n12") %>%
  group_by(Subject) %>%
  summarize(cor_rat_lat = cor(Rating_Score, Latent_Score),
            cor_lat_rank = cor(Latent_Score, Ranking_Score),
            cor_rat_rank = cor(Rating_Score, Ranking_Score))

# show table
liking_stages_outliers
```

```
# A tibble: 2 x 4
  Subject cor_rat_lat cor_lat_rank cor_rat_rank
  <fct>      <dbl>      <dbl>      <dbl>
1 sub-n12    0.313      0.413      0.592
2 sub-n23    0.777     -0.891     -0.783
```

Here we see that *sub-n23* perhaps misunderstood the scale of the ranking task, since the estimated latent liking scores do correlate well with the scores from the rating task. On top of that, the ranking task scores correlate strongly, though negatively, with the latent liking scores. This also suggests that perhaps the scale was misunderstood, and not that our model does not perform well.

The other subject shows moderate correlations across all stages, indicating a more continuously changing preference for paintings, or perhaps inattention or fatigue.

5.2 k-Fold Validity

Finally, we check the result from the k-fold validation procedure.

```
# Test mean accuracy for significance
mean_k_fold_acc <- mean(df_k_fold_acc$Accuracy)
sd_k_fold_acc <- sd(df_k_fold_acc$Accuracy)

# check for significance
k_fold_t_test <- t.test(df_k_fold_acc$Accuracy, mu=0.5)
# Get effect size
k_fold_effect <- cohens_d(df_k_fold_acc$Accuracy, mu=0.5)

# Plot histogram of accuracies
k_fold_plot <- df_k_fold_acc %>% ggplot(aes(x=Accuracy)) +
  geom_histogram(binwidth = 0.025, color='black', fill='cyan') +
  #geom_density(aes(y=after_stat(density) * nrow(df_k_fold_acc) * 0.025),
  ↪ color='red') +
  geom_vline(aes(xintercept = mean_k_fold_acc), color='red', linetype =
  ↪ 'dashed', linewidth = 1) +
  theme_classic() +
  ylab("Number of Observations") +
  xlab("Prediction Accuracy") +
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  theme(
    axis.title = element_text(size = 16),      # Change axis title size
    axis.text = element_text(size = 14))

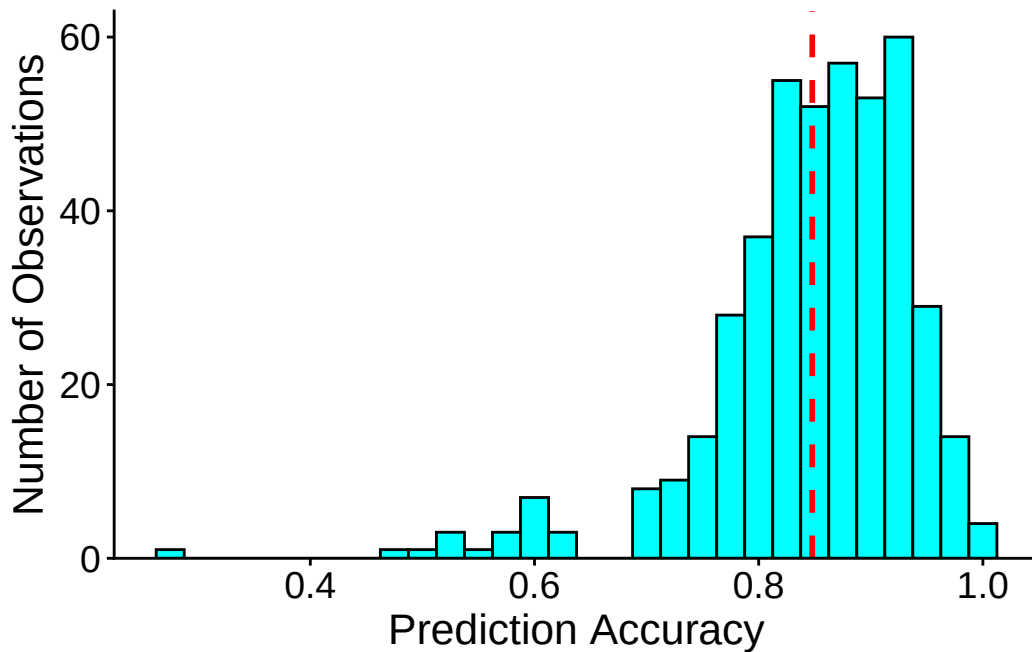
# Store figures
ggsave(filename = "k_fold_validation_plot.png", plot=k_fold_plot,
  ↪ path=figures_path, units="px", dpi=300)
```

Saving 1650 x 1050 px image

```
fig_arranged <- ggarrange(pred_val_plot, k_fold_plot,
                           labels=c("A", "B"), font.label= list(size=16))
ggsave(plot = fig_arranged, filename = "pred_and_kfold_validation.png",
        path=figures_path, units="px", dpi=300)
```

Saving 1650 x 1050 px image

```
# Show the plot
k_fold_plot
```



The figure suggests an overall great performance of the model, with the accuracy differing significantly from chance and a large effect size ($M = 0.85$, $SD = 0.09$), $t(439) = 79.83$, $p < 0.001$, Cohen's $d = 3.81$. This suggests that the model is generalizable, as it is able to predict the painting preference for new trials.

Conclusion

Overall, we found that the model is very capable in predicting a subject's ranking of paintings from least to most liked. On top of that, the k-fold validation procedure showed that the

model is generalizable, in that it can predict which painting will be preferred when shown unseen trials.

This confirms the model’s validity as a tool to estimate a subject’s latent liking scores based on incomplete preference data gathered from the 1-back MRI task.